Блок 1. Архитектура SPA и роутинг

Кейс 01. Зацикливание истории браузера

Причина: `pushState()` добавляет новую запись в историю браузера. При нажатии «Назад» происходит повторный редирект.

Решение: Использовать `replaceState()`, который заменяет текущую запись.

```
function checkAuthAndNavigate(isAuthenticated, currentPath) {
  if (!isAuthenticated && currentPath !== "/login") {
    window.history.replaceState({}, "", "/login");
    renderLoginPage();
  }
}
```

Кейс 02. Утечка памяти в SPA

Причина: Обработчик `resize` продолжает работать даже после удаления шапки. Старые обработчики накапливаются.

Решение: Удалять обработчик при размонтировании компонента.

```
function mountHeader() {
  const header = document.getElementById("header");

  const handleResize = () => {
    header.className = window.innerWidth < 768
      ? "header-mobile"
      : "header-desktop";
  };

  window.addEventListener("resize", handleResize);
  handleResize();

  return () => {
    window.removeEventListener("resize", handleResize);
  };
}
```

Кейс 03. Hard Reload вместо SPA-перехода

Причина: Браузер выполняет стандартный переход по ссылке после обработки `onclick`.

Решение: Использовать `preventDefault()`.

```
function ProductCard({ product, router }) {
  return `
    <div class="card">
      <h3>${product.title}</h3>
      <a href="/cart" onclick="event.preventDefault();
router.navigate('/cart')">
        В корзину
      </a>
    </div>
  `;
}
```

Блок 2. Окружение, зависимости и бандлинг

Кейс 04. Неправильная классификация зависимостей

Причина: React и date-fns нужны приложению при работе, поэтому должны находиться в `dependencies`. Vite нужен для разработки и сборки.

Решение:

```
{
  "dependencies": {
    "react": "^18.3.1",
    "react-dom": "^18.3.1",
    "date-fns": "^3.6.0"
  },
  "devDependencies": {
    "vite": "^5.4.0"
  }
}
```

Кейс 05. Недетерминированная сборка

Причина: Символ `^` разрешает устанавливать более новые совместимые версии пакетов. Поэтому при повторной установке зависимости могут отличаться.

Решение: Использовать `npm ci`, который устанавливает зависимости строго по `package-lock.json`.

```
git pull origin main
npm ci
npm run build
```

Кейс 06. Импорт Node.js модулей в браузер

Причина: Модули `fs` и `path` работают в Node.js, но недоступны в браузере.

Решение: Передавать версию приложения через `vite.config.js`.

```
import { defineConfig } from "vite";
import pkg from "./package.json";

export default defineConfig({
  define: {
    __APP_VERSION__: JSON.stringify(pkg.version)
  }
});
```

В компоненте:

```
export function Footer() {
  return `<footer>Версия: ${__APP_VERSION__}</footer>`;
}
```

Блок 3. Модель памяти, ссылки и мутации

Кейс 07. Мутация данных через ссылку

Причина: Метод `filter()` создаёт новый массив, но объекты внутри него остаются ссылками на исходные объекты.

Решение: Создавать копии объектов через Spread.

```
function resetAttemptsForActiveUsers(usersList) {
  return usersList
    .filter(user => user.isActive)
    .map(user => ({
      ...user,
      loginAttempts: 0
    }));
}
```

Кейс 08. Поверхностное копирование объекта

Причина: Spread копирует только первый уровень объекта. Вложенный объект `security` остаётся общей ссылкой.

Решение: Создать копии вложенных объектов и массива.

```
const userCopy = {
  ...userProfile,
  security: {
    ...userProfile.security,
    roles: [...userProfile.security.roles, "superadmin"]
  }
};
```

Кейс 09. Замыкание переменной var

Причина: `var` имеет функциональную область видимости. Все функции используют одну переменную `i`, которая после цикла равна 6.

Первый способ — использовать let:

```
function createPaginationButtons(pagesCount) {
  const buttons = [];

  for (let i = 1; i <= pagesCount; i++) {
    buttons.push(() => {
      console.log(`Переход на страницу: ${i}`);
    });
  }

  return buttons;
}
```

Второй способ — использовать дополнительное замыкание:

```
function createPaginationButtons(pagesCount) {
  const buttons = [];

  for (var i = 1; i <= pagesCount; i++) {
    buttons.push((page) => () => {
      console.log(`Переход на страницу: ${page}`);
    }(i));
  }

  return buttons;
}
```

Кейс 10. Потеря контекста this

Причина: При передаче метода отдельно от объекта теряется контекст `this`. Поэтому значение `prefix` становится недоступным.

Три способа исправления:

1. Использовать bind:

```
document
  .getElementById("checkout-btn")
  .addEventListener("click",
analyticsTracker.sendEvent.bind(analyticsTracker));
```

2. Использовать стрелочную функцию:

```
document
  .getElementById("checkout-btn")
  .addEventListener("click", (event) => {
    analyticsTracker.sendEvent(event.type);
  });
```

3. Использовать call внутри обработчика:

```
document
  .getElementById("checkout-btn")
  .addEventListener("click", function(event) {
    analyticsTracker.sendEvent.call(analyticsTracker
event.type);
  });
```

Блок 4. Методы массивов

Кейс 11. Мутирующий метод sort()

Причина: Метод `sort()` изменяет исходный массив, нарушая работу общего каталога.

Решение: Создать копию массива перед сортировкой.

```
function getTopExpensiveProducts(products) {
  return [...products]
    .sort((a, b) => b.price - a.price)
    .slice(0, 3);
}
```

Методы `push()`, `pop()`, `shift()`, `unshift()`, `splice()` и `sort()` изменяют исходный массив. Методы `map()`, `filter()`, `find()` и `slice()` его не изменяют.

Кейс 12. Использование forEach вместо map

Причина: `forEach()` не создаёт новый массив, поэтому приходится вручную создавать переменную и заполнять её.

Решение:

```
function mapIdsToTasks(ids) {
  return ids.map(id => ({
    taskId: id,
    isDone: false
  }));
}
```

`map()` предпочтительнее в React, потому что позволяет декларативно создавать новые массивы без изменения исходных данных.

Кейс 13. Проверка falsy-значений через find()

Причина: Число `0` считается ложным значением (`falsy`), поэтому проверка `!targetCode` работает неправильно.

Решение:

```
const exitCodes = [0, 5, 8, 12];

const targetCode = exitCodes.find(code => code === 0);

if (targetCode !== undefined) {
  console.log("Код успешно найден:", targetCode);
} else {
  console.log("Код 0 не найден в списке!");
}
```

Кейс 14. Неинициализированный аккумулятор reduce()

Причина: Без начального значения `reduce()` использует первый объект массива как аккумулятор. При сложении получается некорректный результат.

Решение: Указать начальное значение `0`.

```
const cartItems = [
  { id: 1, title: "Мышь", price: 2000 },
  { id: 2, title: "Коврик", price: 1000 }
];

const totalCost = cartItems.reduce(
  (acc, item) => acc + item.price,
  0
);

console.log(totalCost); // 3000
```

Блок 5. Деструктуризация, Rest и вычисляемые свойства

Кейс 15. Динамическое обновление формы

Причина: `fieldName` создаёт обычное поле с таким названием, а не использует значение переменной.

Решение: Использовать вычисляемое свойство `[fieldName]`.

```
let formState = {
  name: "Иван",
  email: "ivan@test.com"
};

function onFieldChange(fieldName, value) {
  formState = {
    ...formState,
    [fieldName]: value
  };
}
```

Кейс 16. Деструктуризация null

Причина: Значение по умолчанию применяется при `undefined`, но не защищает от `null`.

Решение: Использовать оператор `||` перед деструктуризацией.

```
function renderUserStats(serverData) {
  const { stats = { views: 0, likes: 0 } } = serverData ||
{};

  return `Просмотры: ${stats.views}`;
}

renderUserStats(null);
```

Кейс 17. Неправильный порядок Spread

Причина: Значения, указанные после Spread, перезаписывают предыдущие. Поэтому `type: "text"` заменяет переданный тип пароля.

Решение: Сначала задать значения по умолчанию, затем пропсы. Классы объединить отдельно.

```
function TextInput(props) {
  const mergedProps = {
    type: "text",
    ...props,
    className: `form-input ${props.className ||
""}`.trim()
  };

  return mergedProps;
}

const passwordConfig = TextInput({
  type: "password",
  id: "pass"
});
```

Блок 6. Асинхронный JavaScript, Promise и Race Conditions

Кейс 18. Проглоченная ошибка внутри Promise

Причина: Ошибка внутри обработчика `onload` возникает уже после выполнения конструктора Promise. Поэтому она автоматически не передаётся в `reject()`.

Решение: Использовать `try...catch`.

```
function loadCustomScript(src) {
  return new Promise((resolve, reject) => {
    const script = document.createElement("script");
    script.src = src;

    script.onload = () => {
      try {
        const data = JSON.parse("НЕВАЛИДНЫЙ_JSON");
        resolve(data);
      } catch (error) {
        reject(error);
      }
    };

    script.onerror = () => reject(
      new Error("Ошибка загрузки скрипта")
    );

    document.head.appendChild(script);
  });
}
```

Кейс 19. Race Condition при поиске

Причина: Запросы выполняются параллельно и могут возвращаться в разном порядке. Старый запрос способен перезаписать новые результаты.

Решение: Использовать `AbortController` для отмены предыдущего запроса.

```
let controller;

async function onSearchInput(query) {
  if (controller) {
    controller.abort();
  }

  controller = new AbortController();

  try {
    const response = await fetch(
      `/api/search?q=${encodeURIComponent(query)}`,
      { signal: controller.signal }
    );

    if (!response.ok) {
      throw new Error("Ошибка поиска");
    }

    const data = await response.json();
    renderDropdown(data);

  } catch (error) {
    if (error.name !== "AbortError") {
      console.error(error);
    }
  }
}
```

Кейс 20. Ошибка Promise.all

Причина: `Promise.all()` работает по принципу Fail-Fast. Если хотя бы один запрос завершается ошибкой, весь Promise отклоняется.

Решение: Использовать `Promise.allSettled()`, чтобы получить результат каждого запроса независимо.

```
async function loadFullProfilePage(userId) {
  const results = await Promise.allSettled([
    fetchUserData(userId),
    fetchUserBalance(userId),
    fetchPromoBanner()
  ]);

  const user = results[0].status === "fulfilled"
    ? results[0].value : null;

  const balance = results[1].status === "fulfilled"
    ? results[1].value : null;

  const promo = results[2].status === "fulfilled"
    ? results[2].value : null;

  if (!user || balance === null) {
    throw new Error("Не удалось загрузить основные данные");
  }

  renderProfile({ user, balance, promo });
}
```